nabia web UI: Agents, MCP Servers and Web Application

Running the risk budgeting strategy with AI agents

nabia documentation

October 2026

Abstract

This document is the reference for the web UI project of nabia (`webui/`):

- the AI layer: the three agents with their complete prompts, the MCP servers with every tool's description and input schema, the agent loop, how decisions become positions, and how news sentiment and relevance are produced and aggregated;
- the web application (FastAPI back end, JavaScript front end) and its job runner.

Every computation goes through the `nabia` package of the Python project, which runs the C++ engines of the `nabia_python_bridge` wheel; their references are in `python/doc` and `cpp/doc`. Excerpts, prompts, tool descriptions and configuration fields are *extracted from the source when the document is built*, so they are the text the programs actually use.

Contents

I Overview **3**

1 Purpose **3**

2 Architecture **3**
2.1 Three projects . . . 3
2.2 Directory map . . . 4
2.3 Environment . . . 4

II The AI layer **4**

3 Agents: roles, data flow and configuration **5**
3.1 Configuration . . . 6

4 MCP servers and tool catalogue **6**
4.1 Design . . . 6
4.2 Catalogue . . . 8
4.3 Server `nabia-news` . . . 8
4.4 Server `nabia-stats` . . . 9
4.5 Server `nabia-portfolio` . . . 10

5 The agent runtime **11**
5.1 Starting an agent . . . 11
5.2 The loop . . . 12
5.3 Providers . . . 14

6 Agent prompts **16**
6.1 Structure and templating 16
6.2 News analyzer 16
6.3 Stats analyzer 17
6.4 Execution agent 17

7 From decisions to positions **19**
7.1 Budget normalization within bounds 19
7.2 Target weights 20
7.3 The trend rule baseline 21
7.4 Backtest mode 21

8 News: collection, sentiment and relevance **22**
8.1 Collection 22
8.2 How sentiment and relevance are produced 23
8.3 Aggregation 24
8.4 Use and limitations 25

III Operation **26**

9 Web application and job runner **26**

10 Verification **27**

11 Limitations and future work **27**

Part I
Overview

1 Purpose

The web UI runs nabia's strategy day to day: agents read the news and the statistics, decide the risk budgets, and the C++ engine turns them into positions; the web application starts these jobs and shows every decision. The central design decision is to separate *who decides the risk budgets* from *how budgets become positions*:

- Budgets are decided by a fixed rule, by equal risk, or by AI agents.
- Positions come from a tested numerical engine (`nabia.latest_weights` and `nabia.backtest`).

Agents never set weights. They choose budgets within bounds, and the engine computes the weights, so an agent's influence is bounded and every decision is reproducible from its stored budgets.

2 Architecture

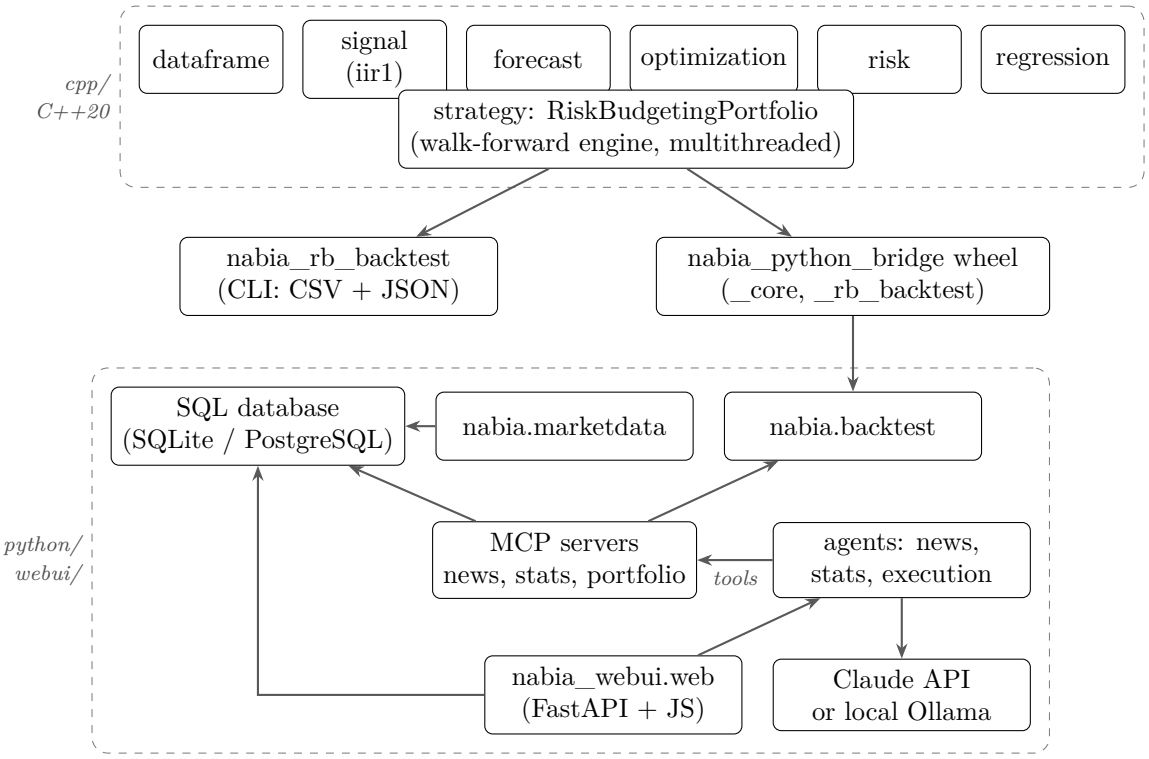


Figure 1: The whole system and its three projects. Arrows point from a component to what it uses.

2.1 Three projects

The repository holds three independent projects. Each opens on its own in an IDE, and each has its own documentation in a `doc` folder:

Folder	IDE	Contents
`cpp/`	CLion (CMake)	the C++ library (`include`, `src`), its tests, the command-line application, and the pybind11 bindings that build the `nabia_python_bridge` wheel into `dist/`
`python/`	PyCharm	the `nabia` package (backtests, analytics, market data) on top of the wheel, the examples, and the Python tests including the bridge tests
`webui/`	PyCharm	`nabia_webui`: the agents, their MCP servers and the web app, on top of the `nabia` package

The projects depend on each other only through installed packages: `cpp` builds the wheel, `python` installs it (`python/scripts/setup_env.sh`), and `webui` installs both the wheel and `python` in editable mode (`webui/scripts/setup_env.sh`). Shared data, the market database `data/market.db`, is at the repository root.

2.2 Directory map

Path (in `webui/`)	Contents
`nabia_webui/ai/agents`	agent runtime, providers (Claude API, Ollama), the agent team
`nabia_webui/ai/servers`	MCP servers: news, stats, portfolio
`nabia_webui/ai`	configuration, database tables, pipeline (daily and backtest), command line
`nabia_webui/web`	FastAPI back end (`app.py`), job runner, static front end
`tests`	pytest suites
`ai_config.json`	configuration (symbols, bounds, model, database URL)
`scripts/setup_env.sh`	creates `.venv` with the wheel, `nabia` and `nabia_webui`
`doc`	this reference (LaTeX source, PDF, `build.sh`)

One SQL database holds prices, news, statistics reports, decisions, agent transcripts, jobs, job logs and backtest runs. Access goes through SQLAlchemy Core with portable types, so moving from SQLite to PostgreSQL changes only the URL. By default it is the shared `../data/market.db`.

2.3 Environment

```
webui/scripts/setup_env.sh                                  # webui/.venv
cd webui && .venv/bin/python -m pytest                      # 24 tests, 1 opt-in (Ollama) skipped
.venv/bin/python -m nabia_webui.web --config ai_config.json   # http://127.0.0.1:8000
.venv/bin/python -m nabia_webui.ai daily --config ai_config.json
```

In PyCharm, open `webui/` and select `webui/.venv` as the interpreter; run configurations should use `webui/` as the working directory, where `ai_config.json` and its relative database path resolve.

Part II
The AI layer

3 Agents: roles, data flow and configuration

Agent	Tools	Output
News analyzer	`news` server; on Claude also web search and web fetch	stored articles with topic, assets, sentiment, relevance
Stats analyzer	`stats` server	a structured report (`StatsReport`) with a budget tilt per asset
Execution agent	`news` and `stats` read-only, `portfolio`	risk budgets with a rationale, then target weights from the C++ engine

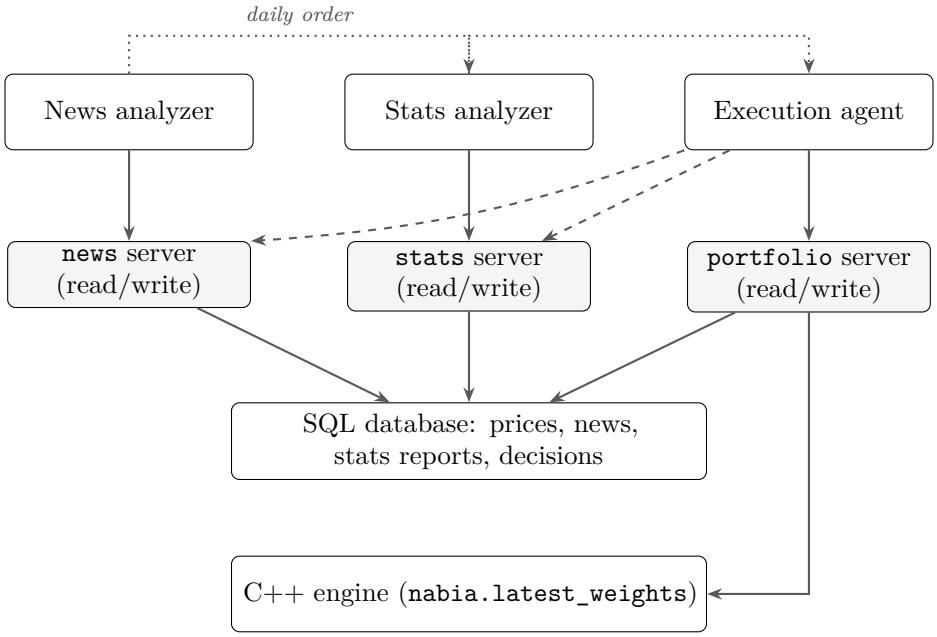


Figure 2: The three agents, the MCP servers they use, and their order in the daily pipeline. Dashed arrows: read-only access (the servers run with `--read-only`).

Pipelines.

- The *daily* pipeline runs news, then stats, then execution, for today.
- The *backtest* pipeline runs stats and execution at each decision date (§7.4).

Every agent run is stored with its turns, tool calls (inputs and outputs, truncated to 2,000 characters), token usage and any error, which gives a complete audit trail.

Code

running and recording an agent — `nabia_webui/ai/agents/team.py:111-125`
daily pipeline — `nabia_webui/ai/pipeline.py:36-50`
database tables — `nabia_webui/ai/db.py:39-120`

3.1 Configuration

Agents, servers and pipeline share one `AIConfig`, saved as JSON so that the MCP server processes read the same settings. Unknown keys are rejected. Validation requires at least two distinct symbols, $0 < b_{\min} \leq 1/n \leq b_{\max} \leq 1$, a known provider, and no `budgets` in `portfolio`, because budgets are what the agents decide.

Field	Default	Meaning
`db_url`	`'sqlite:///market.db'`	Database with market data (nabia.marketdata), news, stats reports and decisions.
`symbols`	the 9 default symbols (*Default AI portfolio universe* (Python reference))	Portfolio universe: one risk budget per symbol.
`portfolio`	*Portfolio parameters* (C++ reference)	nabia.backtest parameters (without budgets, which the execution agent sets).
`min_budget`	`0.02`	Lower bound on each normalized risk budget, enforced by set_risk_budgets.
`max_budget`	`0.4`	Upper bound on each normalized risk budget, enforced by set_risk_budgets.
`provider`	`'anthropic'`	Model provider: "anthropic" (Claude API) or "ollama" (a local model).
`model`	`'claude-opus-5-5'`	Claude model, when provider is "anthropic".
`effort`	`{'news': 'medium', 'stats': 'high', '...`	Claude effort level per agent (news, stats, execution).
`ollama_model`	`'qwen3.8:latest'`	Local model, when provider is "ollama" (it must support tool calling: `ollama show <model>`).
`ollama_host`	`'http://localhost:11434'`	URL of the Ollama server.
`ollama_options`	`{'temperature': 0.2, 'num_ctx': 32768}`	Ollama model options; num_ctx must hold the prompts, tool schemas and tool results.
`ollama_think`	`False`	Let thinking models reason before answering (slower, sometimes better tool use).
`news_feeds`	10 feeds, §8.1	RSS / Atom feeds read by fetch_news_feeds, by source name.
`news_max_searches`	`10`	Cap on web searches per news collection run.
`news_lookback_days`	`7`	Window of stored news the execution agent reviews.
`stats_history_days`	`400`	Calendar days of prices loaded by the stats tools (covers 1y signals).

Code

`AIConfig` — `nabia_webui/ai/config.py:40-104`

4 MCP servers and tool catalogue

4.1 Design

Each server is a standalone MCP server over stdio:

```
python -m nabia_webui.ai.servers.<name> --config ... --run ... [--read-only]
```

The agents use them, and so can any MCP client, for example Claude Code. A server is built in three layers:

- A `ServerContext` holds the configuration, the run name and the two stores (market data and agent data).
- Tools are plain Python functions registered with `@tool(server)`.
- The MCP library derives each tool's *description* from the function's docstring and its *input schema* from the type annotations. The catalogue below is extracted from the same source, so it is exactly what the model sees.

Four rules apply to every tool.

Point in time Analytical tools take `as_of` (`YYYY-MM-DD`) and read only data dated on or before it. A test rewrites every price after `as_of` and checks that every tool's output is unchanged.

Errors the model can act on Anticipated failures, such as wrong symbols or out-of-range values, raise `ValueError` or `KeyError`. The decorator turns them into MCP `ToolError`s carrying the message, so the model sees why a call failed and can correct it. Any other exception would reach the model only as a generic error.

Read-only mode `--read-only` omits the tools that write. The execution agent cannot store news or overwrite statistics reports.

Runs `--run <name>` scopes reports and decisions to a backtest run, or to `live`, so backtests never mix with live decisions.

Code 1: Tool registration
`nabia_webui/ai/servers/common.py:94-114`

```python
def tool(server: MCPServer) -> Callable[[F], F]:
    """Registers a function as an MCP tool, reporting ValueError / KeyError to the
    model as tool errors with their message (MCP otherwise treats them as crashes
    and hides the text), so the agent can correct its call."""

    def decorator(fn: F) -> F:
        @functools.wraps(fn)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            try:
                return fn(*args, **kwargs)
            except (ValueError, KeyError) as error:
                raise ToolError(str(error)) from error

        # Resolved annotations, so schema generation does not depend on this module's globals.
        wrapper.__signature__ = inspect.signature(fn, eval_str=True)  # type: ignore[attr-defined]
        wrapper.__annotations__ = {**{k: p.annotation for k, p in wrapper.__signature__.parameters.
    ↪ items()},
                                   "return": wrapper.__signature__.return_annotation}
        server.tool()(wrapper)
        return fn

    return decorator
```

Code 2: The as-of date
`nabia_webui/ai/servers/common.py:52-60`

```python
def parse_as_of(text: str) -> dt.date:
    """'YYYY-MM-DD' (or 'today'). The as-of date bounds every query: tools never
    return data dated after it, which keeps backtests free of look-ahead."""
    if text == "today":
        return dt.date.today()
    try:
        return dt.date.fromisoformat(text)
    except ValueError as error:
        raise ValueError(f"as_of must be YYYY-MM-DD, got {text!r}") from error
```

Code

`ServerContext` — `nabia_webui/ai/servers/common.py:23-49`
prices up to as_of, universe check — `nabia_webui/ai/servers/common.py:38-49`
JSON cleaning (NaN to null, rounding) — `nabia_webui/ai/servers/common.py:63-79`
news server — `nabia_webui/ai/servers/news.py:42-136`
stats server — `nabia_webui/ai/servers/stats.py:31-105`
portfolio server — `nabia_webui/ai/servers/portfolio.py:16-72`

4.2 Catalogue

The rest of this section is generated from the server sources. For each tool it gives the description shown to the model, whether the tool writes, the source location, and the parameters with their types and defaults. The input schemas of the structured tools follow each server. In the news schemas, `Topic` is one of:

- monetary_policy, inflation, growth, labor, earnings, fiscal, geopolitics;
- energy, metals, agriculture, crypto, rates, credit, fx, regulation, other.

`AssetClass` is one of equity_index, bond_future, commodity and crypto.

4.3 Server `nabia-news`

Module: `nabia.ai.servers.news` (`nabia_webui/ai/servers/news.py`). **Instructions sent to the client:** *Financial news database of the nabia trend portfolio. Dates are YYYY-MM-DD; queries never return articles published after as_of.*

`store_news_articles`

writes; removed in read-only mode — `nabia_webui/ai/servers/news.py:50-68`

Store news articles (upsert by URL, so storing an article twice updates it).

Only store articles with a real URL and publication date you have seen. Symbols must be portfolio symbols (see the news_summary universe).

`articles list[NewsArticle]`, required

`fetch_news_feeds`

writes; removed in read-only mode — `nabia_webui/ai/servers/news.py:71-90`

Latest headlines from the configured financial news RSS feeds (markets, economy, central banks, energy, crypto), newest first, skipping articles already stored. Use it to find candidate stories to classify and store.

`max_age_days int`, default 2

`max_items int`, default 60

`query_news`

read-only — `nabia_webui/ai/servers/news.py:92-99`

Stored articles published in the `days` days up to and including as_of, most recent and relevant first, optionally filtered by topic, symbol or asset class.

`as_of str`, required

`days int`, default 7

`topic Topic | None`, default None

`symbol str | None`, default None

`asset_class AssetClass | None`, default None

`limit int`, default 30

`news_summary`

read-only — `nabia_webui/ai/servers/news.py:101-134`

Aggregate view of the news in the `days` days up to as_of: article counts and relevance-weighted sentiment per topic, asset class and symbol, plus the top headlines.

`as_of str`, required

`days int`, default 7

Input schema `NewsArticle`

pydantic model `nabia_webui/ai/servers/news.py:27-39`

One news item, as extracted by the news analyzer.

`url str`. Canonical URL of the article (the unique key)

`published_date dt.date`. Publication date, YYYY-MM-DD

`title str`

`source str`. Publisher, e.g. Reuters, Bloomberg, FT

`topic Topic`. Main topic

`asset_classes list[AssetClass]`. Asset classes the news affects

`symbols list[str]`. Portfolio symbols directly affected

`sentiment float`. $\geq$ -1.0; $\leq$ 1.0; Price impact on the affected assets: -1 bearish .. +1 bullish

`relevance float`. $\geq$ 0.0; $\leq$ 1.0; Importance for a cross-asset portfolio: 0 .. 1

`summary str`. Two or three factual sentences: what happened and why it matters

4.4 Server `nabia-stats`

Module: `nabia.ai.servers.stats` (`nabia_webui/ai/servers/stats.py`). **Instructions sent to the client:** *Statistics of the nabia portfolio assets. Every tool takes as_of (YYYY-MM-DD) and uses only prices dated on or before it. Volatilities are annualized; returns are simple.*

`list_universe`

read-only `nabia_webui/ai/servers/stats.py:38-55`

Portfolio symbols with their asset class, name and price coverage up to as_of.

`as_of str`, required

`correlation_matrix`

read-only `nabia_webui/ai/servers/stats.py:57-64`

Correlation of daily returns over the last `window` dates where all assets traded, plus the average pairwise correlation.

`as_of str`, required

`window int`, default 60

`rolling_volatility`

read-only `nabia_webui/ai/servers/stats.py:66-71`

Annualized volatility per asset over 20, 60 and 120 days, and the percentile of today's 20-day volatility within its last year (1.0 = highest of the year).

`as_of str`, required

`trend_signals`

read-only `nabia_webui/ai/servers/stats.py:73-78`

Per asset: 1/3/6/12-month returns, distance from the 50/200-day moving averages, volatility-scaled 12-1 month momentum, current and 1-year max drawdown.

`as_of str`, required

`market_regime`

read-only `nabia_webui/ai/servers/stats.py:80-85`

Cross-asset regime: average correlation now vs over a year, median volatility percentile, and the share of assets in an uptrend.

`as_of` `str`, required

`get_stats_report`

read-only `nabia_webui/ai/servers/stats.py:87-91`

The latest stats report saved on or before as_of (null if none).

`as_of` `str`, required

`save_stats_report`

writes; removed in read-only mode `nabia_webui/ai/servers/stats.py:94-103`

Save the structured analysis for as_of (replaces an earlier one for the same date). Include every portfolio symbol in `assets`.

`as_of` `str`, required

`report` `StatsReport`, required

Input schema `AssetView`

pydantic model `nabia_webui/ai/servers/stats.py:15-21`

`symbol` `str`

`trend` `str`. e.g. strong up, up, flat, down, strong down

`volatility_regime` `str`. e.g. calm, normal, elevated, stressed

`diversification` `str`. How it correlates with the rest of the portfolio

`budget_tilt` `float`. $\geq$ -1.0; $\leq$ 1.0; Suggested tilt of its risk budget: -1 cut .. +1 raise

`reason` `str`

Input schema `StatsReport`

pydantic model `nabia_webui/ai/servers/stats.py:24-28`

`summary` `str`. Cross-asset picture in a few sentences

`regime` `str`. e.g. risk-on trending, risk-off, high-correlation stress, choppy

`assets` `list[AssetView]`

`key_risks` `list[str]`

4.5 Server `nabia-portfolio`

Module: `nabia.ai.servers.portfolio` (`nabia_webui/ai/servers/portfolio.py`). **Instructions sent to the client:** *Risk budgets and target weights of the nabia risk budgeting portfolio. Budgets are each asset's share of portfolio risk; they are normalized to sum to 1.*

`get_portfolio_config`

read-only `nabia_webui/ai/servers/portfolio.py:23-29`

Universe, budget bounds and the portfolio engine settings (covariance window, rebalancing, volatility target, costs). *No parameters.*

get_previous_decision

read-only nabia_webui/ai/servers/portfolio.py:31-35

The most recent risk budget decision strictly before as_of (null if none), with its rationale and resulting weights.

as_of **str**, required

preview_budgets

read-only nabia_webui/ai/servers/portfolio.py:37-42

Show how budgets would be normalized and bounded, without saving anything.

budgets **dict[str, float]**, required

set_risk_budgets

writes; removed in read-only mode nabia_webui/ai/servers/portfolio.py:45-58

Decide the risk budgets for as_of: one positive number per portfolio symbol (relative sizes; normalized to sum to 1 and bounded to [min_budget, max_budget]). The rationale should cite the evidence (stats, news) behind each tilt.

as_of **str**, required

budgets **dict[str, float]**, required

rationale **str**, required

compute_target_weights

writes; removed in read-only mode nabia_webui/ai/servers/portfolio.py:60-70

Target portfolio weights from the budgets decided for as_of: risk budgeting on the trailing covariance (C++ engine), scaled to the volatility target. Call it after set_risk_budgets.

as_of **str**, required

5 The agent runtime

5.1 Starting an agent

`AgentRuntime.run` starts the agent's MCP servers as subprocesses. They use the same Python interpreter and module path, with `--config` pointing to the saved configuration and `--run` set to the run name. The runtime then:

- initializes a client session per server;
- lists the tools of every session and rejects name collisions between servers;
- adds the hosted tools, if any (web search and fetch on Claude);
- runs the loop.

Errors raised inside the loop are re-raised only after the sessions close cleanly. Otherwise the MCP task groups would wrap them in an `ExceptionGroup`.

Code

run: servers, tools, loop — nabia_webui/ai/agents/runtime.py:160-190
server subprocess parameters — nabia_webui/ai/agents/runtime.py:151-158
client selection by provider — nabia_webui/ai/agents/runtime.py:133-145

5.2 The loop

The runtime uses a manual tool-use loop, not an SDK tool runner. One agent talks to several MCP processes at once, mixes their tools with hosted tools, and records every call. Each turn proceeds as follows.

1. **Request.** One request carries the system prompt, the tools and the conversation so far, plus these options:
   - `max_tokens`: 16,000;
   - adaptive thinking;
   - the agent's effort level;
   - prompt caching of the stable prefix (tools, system prompt, history);
   - server-side refusal fallbacks.
2. **Streaming.** The reply is *streamed*. Text and thinking deltas go to the log as they arrive, so a job's log follows the model live (§9).
3. **Stop reasons.**
   - `refusal` raises an error naming the category.
   - `pause_turn` (a long hosted-tool turn) resends the conversation so the model can resume.
   - `max_tokens` is an error.
   - Any other reason that is not `tool_use` ends the run with the final text.
4. **Tools.** All `tool_use` blocks of the turn run concurrently. Their results return *in one user message*. A failing tool, or a crashed server, becomes an error result, so the agent can recover.

A run is limited to 40 turns.

Code 3: The agent loop
`nabia_webui/ai/agents/runtime.py:192-238`

```
async def _loop(self, record: AgentRun, system: str, prompt: str, tools: list[dict[str, Any]],
                sessions: dict[str, ClientSession], effort: str) -> None:
    messages: list[dict[str, Any]] = [{"role": "user", "content": prompt}]
    for turn in range(1, self.max_turns + 1):
        log.info("%s: turn %d, waiting for the model...", record.agent, turn)
        started = time.monotonic()
        live = LiveOutput(record.agent)
        response = await self._create(live, dict(
            model=self.config.model,
            max_tokens=MAX_TOKENS,
            system=system,
            tools=tools,
            messages=messages,
            thinking={"type": "adaptive"},
            output_config={"effort": effort},
            cache_control={"type": "ephemeral"},  # caches the stable prefix: tools, system, history
            betas=["server-side-fallback-2026-07-01"],
            fallbacks="default",  # re-run a refused request on a fallback model
        ))
        record.turns += 1
        self._add_usage(record, response.usage)
        log.info("%s: turn %d -> %s (%s in / %s out tokens, %.1fs)", record.agent, turn, response.
    ↪ stop_reason,
                 getattr(response.usage, "input_tokens", "?"), getattr(response.usage, "output_tokens",
    ↪  "?"),
                 time.monotonic() - started)
        for block in response.content:
```

```
            if block.type == "text" and block.text.strip() and not live.chars and response.stop_reason
    ↪ == "tool_use":
                log.info("%s says: %s", record.agent, _preview(block.text, 300))  # not streamed
            elif block.type == "server_tool_use":
                log.info("%s -> %s %s", record.agent, block.name, _preview(getattr(block, "input", {}))
    ↪ )
        if response.stop_reason == "refusal":
            details = getattr(response, "stop_details", None)
            raise AgentRefusedError(f"{record.agent} refused: {getattr(details, 'category', None)}")
        messages.append({"role": "assistant", "content": response.content})
        record.server_tool_calls += sum(1 for b in response.content if b.type == "server_tool_use")
        if response.stop_reason == "pause_turn":  # a long server-tool turn: resend to resume it
            continue
        if response.stop_reason == "max_tokens":
            raise RuntimeError(f"{record.agent}: response hit max_tokens ({MAX_TOKENS})")
        if response.stop_reason != "tool_use":
            record.final_text = "\n".join(b.text for b in response.content if b.type == "text").strip()
            log.info("%s finished after %d turns, %d tool calls: %s", record.agent, record.turns,
                     len(record.tool_calls), _preview(record.final_text, 400))
            return
        calls = [b for b in response.content if b.type == "tool_use"]
        results = await asyncio.gather(*(self._call_tool(record, sessions, b) for b in calls))
        messages.append({"role": "user", "content": list(results)})  # all results in one message
    raise RuntimeError(f"{record.agent}: no final answer after {self.max_turns} turns")
```

Code 4: One streamed model turn

`nabia_webui/ai/agents/runtime.py:240-256`

```
async def _create(self, live: LiveOutput, request: dict[str, Any]) -> Any:
    """One model turn, streamed when the client supports it, so the output is
    logged as it is written; clients with only ‘‘create‘‘ (tests) are awaited whole."""
    messages_api = self.client.beta.messages
    if not hasattr(messages_api, "stream"):
        return await messages_api.create(**request)
    async with messages_api.stream(**request) as stream:
        async for event in stream:
            if event.type == "content_block_delta":
                delta = event.delta
                if delta.type == "text_delta":
                    live.add("text", delta.text)
                elif delta.type == "thinking_delta":
                    live.add("thinking", delta.thinking)
        response = await stream.get_final_message()
    live.flush()
    return response
```

Code 5: Live logging of streamed output

`nabia_webui/ai/agents/runtime.py:74-111`

```
class LiveOutput:
    """Logs a streamed reply while it is written: a line per paragraph, or every
    few seconds / few hundred characters for long lines, so a job log follows
    the model as it thinks and writes."""

    def __init__(self, agent: str, max_chars: int = 400, max_seconds: float = 3.0) -> None:
        self.agent = agent
        self.max_chars = max_chars
        self.max_seconds = max_seconds
        self.kind = ""
        self.buffer = ""
        self.last = time.monotonic()
        self.chars = 0

    def add(self, kind: str, text: str) -> None:
        if kind != self.kind:
            self.flush()
            self.kind = kind
        self.buffer += text
        self.chars += len(text)
        while "\n" in self.buffer:
            line, self.buffer = self.buffer.split("\n", 1)
```

```
            self._emit(line)
        slow = len(self.buffer) >= self.max_chars // 5 and time.monotonic() - self.last >= self.
    ↪ max_seconds
        if len(self.buffer) >= self.max_chars or slow:  # long lines in pieces; slow streams every few
    ↪ seconds
            self._emit(self.buffer)
            self.buffer = ""

    def flush(self) -> None:
        if self.buffer:
            self._emit(self.buffer)
        self.buffer = ""

    def _emit(self, text: str) -> None:
        self.last = time.monotonic()
        if text.strip():
            label = "thinking" if self.kind == "thinking" else "writes"
            model_log.info("%s %s: %s", self.agent, label, text.strip())
```

Code 6: Executing one tool call on its MCP server
`nabia_webui/ai/agents/runtime.py:258-276`

```
async def _call_tool(self, record: AgentRun, sessions: dict[str, ClientSession], block: Any) -> dict[
    ↪ str, Any]:
    session = sessions.get(block.name)
    log.info("%s -> %s(%s)", record.agent, block.name, _preview(dict(block.input)))
    try:
        if session is None:
            raise ValueError(f"unknown tool {block.name}")
        result = await session.call_tool(block.name, dict(block.input))
        text = "\n".join(c.text for c in result.content if getattr(c, "type", "") == "text")
        if not text and getattr(result, "structured_content", None) is not None:
            text = json.dumps(result.structured_content)
        is_error = bool(result.is_error)
    except Exception as error:  # a broken server must not kill the agent: report it as a tool error
        text, is_error = f"{type(error).__name__}: {error}", True
    record.tool_calls.append(ToolCall(block.name, dict(block.input), is_error, text[:2000]))
    if is_error:
        log.warning("%s <- %s failed: %s", record.agent, block.name, _preview(text, 300))
    else:
        log.info("%s <- %s: %s", record.agent, block.name, _preview(text))
    return {"type": "tool_result", "tool_use_id": block.id, "content": text or "(no output)", "is_error
    ↪ ": is_error}
```

5.3 Providers

Claude (`provider: "anthropic"`) uses `claude-opus-5-5` through the Anthropic SDK with the options above.

Local model (`provider: "ollama"`) uses an adapter that exposes the same interface (`beta.messages.create` and `.stream`) and translates to Ollama's chat API:

- the system prompt becomes a system message, and tools become function tools;
- assistant `tool_use` blocks become `tool_calls`, and `tool_result` blocks become `tool` messages (errors prefixed `ERROR:`);
- streamed NDJSON chunks become text and thinking deltas, and the merged reply becomes the final message;
- Claude-only options are ignored, and hosted tools are dropped, so local news comes from RSS.

Model options (temperature 0.2, a 32k context) come from the configuration.

Code 7: Ollama streaming as Anthropic-style events
`nabia_webui/ai/agents/ollama.py:50-80`

```
class _Stream:
    """A streamed reply: an async context manager and iterator of delta events."""

    def __init__(self, owner: OllamaClient, kwargs: dict[str, Any]) -> None:
        self._owner = owner
        self._kwargs = kwargs
        self._final: SimpleNamespace | None = None

    async def __aenter__(self) -> _Stream:
        return self

    async def __aexit__(self, *exc: Any) -> None:
        return None

    async def __aiter__(self) -> AsyncIterator[SimpleNamespace]:
        data: dict[str, Any] = {}
        async for chunk in self._owner._chat(self._kwargs, stream=True):
            message = chunk.get("message") or {}
            for kind, key in (("thinking", "thinking"), ("text", "content")):
                if message.get(key):
                    yield SimpleNamespace(type="content_block_delta",
                                          delta=SimpleNamespace(type=f"{kind}_delta", **{kind: message[
    ↪ key]}))
            _merge(data, chunk)
        self._final = self._owner._response(data)

    async def get_final_message(self) -> SimpleNamespace:
        if self._final is None:
            async for _ in self:
                pass
        assert self._final is not None
        return self._final
```

Code 8: Ollama replies translated to content blocks
`nabia_webui/ai/agents/ollama.py:183-209`

```
def _response(self, data: dict[str, Any]) -> SimpleNamespace:
    message = data.get("message", {})
    content: list[SimpleNamespace] = []
    text = (message.get("content") or "").strip()
    if text:
        content.append(SimpleNamespace(type="text", text=text))
    for call in message.get("tool_calls") or []:
        function = call.get("function", {})
        arguments = function.get("arguments") or {}
        if isinstance(arguments, str):  # some models return a JSON string
            try:
                arguments = json.loads(arguments)
            except json.JSONDecodeError:
                arguments = {"_unparsed": arguments}
        call_id = call.get("id") or f"call_{next(self._ids)}"
        self._tool_names[call_id] = function.get("name", "")
        content.append(SimpleNamespace(type="tool_use", id=call_id, name=function.get("name", ""),
    ↪ input=arguments))
    if any(b.type == "tool_use" for b in content):
        stop_reason = "tool_use"
    elif data.get("done_reason") == "length":
        stop_reason = "max_tokens"
    else:
        stop_reason = "end_turn"
    usage = SimpleNamespace(input_tokens=data.get("prompt_eval_count", 0), output_tokens=data.get("
    ↪ eval_count", 0),
                            cache_read_input_tokens=0, cache_creation_input_tokens=0)
    return SimpleNamespace(content=content, stop_reason=stop_reason, usage=usage, stop_details=None,
                           model=data.get("model", self.model))
```

Code

request translation — `nabia_webui/ai/agents/ollama.py:119-153`

HTTP call, streamed or not — `nabia_webui/ai/agents/ollama.py:162-181`

6 Agent prompts

6.1 Structure and templating

Every system prompt has the same five parts:

1. the agent's role in a cross-asset trend portfolio;
2. the *as-of rule*;
3. the portfolio universe;
4. the job;
5. a numbered workflow that names the tools to use, ending with the expected output.

`AgentTeam._fill` fills the templates:

`{as_of}` the decision date. In live mode it is today.

`{universe}` one line per portfolio symbol, `- SYMBOL: name, asset_class`, from the asset table.

`{as_of_rule}` the rule below, with the date filled in.

`{news_days}` the news window of the execution agent (7 by default).

`{research}` the news analyzer's research step. With web access (Claude) it adds web search and fetch. With a local model it uses the RSS headlines only.

`{mode_note}` the backtest note for non-live runs, and empty otherwise.

Code 9: The as-of rule (in every system prompt)
`nabia_webui/ai/agents/team.py:17-20`

```
Today is {as_of}. Treat that date as the present: base every judgement on what the tools return (they
    ↪ only see data up to {as_of}) and never on knowledge of events after {as_of}.
```

Code 10: Template filling
`nabia_webui/ai/agents/team.py:135-139`

```
def _fill(self, template: str, as_of: dt.date, **extra: Any) -> str:
    values = {"as_of": as_of.isoformat(), "universe": self._universe(),
              "news_days": self.config.news_lookback_days, **extra}
    values["as_of_rule"] = AS_OF_RULE.format(**values)
    return template.format(**values)
```

Code

the universe lines — `nabia_webui/ai/agents/team.py:127-133`

6.2 News analyzer

The news analyzer runs with effort `medium`, the `news` server (writable) and, on Claude, web search and web fetch (each capped at `news_max_searches` uses). Its user message is *"Collect and store the market-moving news for {as_of}."* The prompt defines the sentiment and relevance rubric (§8) and requires real URLs and dates.

Code 11: News analyzer system prompt (template)
`nabia_webui/ai/agents/team.py:22-43`

```
You are the news analyst of a systematic cross-asset trend portfolio.

{as_of_rule}

Portfolio universe (symbol: name, asset class):
{universe}

Your job: find the financial news published on {as_of} and the previous day that matters for these
assets, and store it in the news database. Cover central banks and rates, inflation and growth data,
equity markets, energy and metals, and crypto; prefer primary and reputable sources.

How to work:
1. Call query_news for {as_of} (days=2) to see what is already stored; do not store it again.
2. {research}
   Aim for 8-20 distinct, material articles, not every headline: skip stories with no plausible
   effect on these assets.
3. Store them with store_news_articles (in batches is fine). Use only URLs and dates you actually saw
   in the feed items or search results; never invent them. topic: the main driver. asset_classes and
     ↪ symbols: what the story
   moves (symbols only from the universe above). sentiment: expected price impact on the affected
   assets, -1 clearly bearish to +1 clearly bullish (0 neutral or mixed). relevance: 1 for
   portfolio-wide shocks, 0.5 for one asset class, lower for minor items.
4. Finish with five lines summarizing the dominant themes and their direction.
```

6.3 Stats analyzer

The stats analyzer runs with effort `high` and the `stats` server (writable). Its user message is *"Analyze the portfolio assets as of {as_of} and save the stats report."* It must:

- judge each asset's trend, volatility regime and diversification;
- propose a tilt in $[-1, 1]$ proportionate to the evidence;
- save a schema-validated report (`StatsReport`) covering every symbol.

Code 12: Stats analyzer system prompt (template)
`nabia_webui/ai/agents/team.py:45-66`

```
You are the quantitative analyst of a systematic cross-asset trend portfolio that allocates
risk across the assets below with risk budgets (each asset's share of portfolio risk).

{as_of_rule}

Portfolio universe:
{universe}

Your job: analyze the statistical state of these assets as of {as_of} and save a structured report
for the portfolio manager.

How to work:
1. Look at market_regime, trend_signals, rolling_volatility and correlation_matrix (60-day, and
   252-day for context). Use get_stats_report to see the previous report for continuity.
2. For each asset judge: trend (sign, strength and persistence across horizons), volatility regime
   (level and percentile vs its last year), and diversification (correlation with the others,
   clusters of assets that move together).
3. Suggest a budget tilt per asset in [-1, 1]: positive for persistent uptrends with contained
   volatility and diversification value; negative for downtrends, stressed volatility or redundant
   exposure. Tilts should be proportionate to the strength of the evidence; 0 when unclear.
4. Save the report with save_stats_report (every symbol in assets, numbers cited in each reason).
5. Finish with a short summary of the regime and the main tilts.
```

6.4 Execution agent

The execution agent runs with effort `high`, the `news` and `stats` servers read-only, and the `portfolio` server. Its user message is *"Decide the risk budgets for {as_of} and compute the*

target weights.” The prompt fixes its decision policy:

- start from equal risk;
- tilt mainly on the statistical evidence, using news as a secondary input;
- respect the bounds;
- keep budgets close to the previous decision because trading costs money, and explain any large change.

Code 13: Execution agent system prompt (template)
`nabia_webui/ai/agents/team.py:68-91`

```
You are the portfolio manager of a systematic cross-asset trend portfolio. The portfolio holds
the assets below with long-only risk budgeting: you set each asset's risk budget (its share of total
portfolio risk); the engine turns budgets into weights using the trailing covariance and scales the
portfolio to its volatility target.

{as_of_rule}

Portfolio universe:
{universe}

Your job: decide the risk budgets for {as_of} and compute the resulting target weights.

How to work:
1. Read get_portfolio_config (budget bounds, settings) and get_previous_decision.
2. Read the quantitative view with get_stats_report and its underlying signals if needed.
3. Read news_summary for {as_of} (last {news_days} days) and query_news for detail on material themes.
4. Decide budgets. Start from equal risk and tilt with evidence: primarily the statistical trend and
   risk picture; news as a secondary input that can confirm, temper or flag a risk the statistics do
   not show yet. Respect the bounds. Trading has costs: keep budgets close to the previous decision
   unless the evidence has changed materially, and explain any large change.
5. Call set_risk_budgets for {as_of} with a rationale that cites the evidence behind each tilt, then
   compute_target_weights for {as_of}.
6. Finish with a table of budgets and weights and a three-sentence rationale.
{mode_note}
```

Code 14: Backtest note, appended for non-live runs (`{mode_note}`)
`nabia_webui/ai/agents/team.py:93-95`

```
This is a historical simulation: news is only what was stored before {as_of}, and there is no web
access. Decide exactly as you would have on {as_of}.
```

Code 15: How each agent is called: prompts, user messages, servers, effort
`nabia_webui/ai/agents/team.py:141-177`

```
async def collect_news(self, as_of: dt.date | None = None) -> AgentRun:
    """News analyzer: RSS feeds (and web search on Claude) -> news database.
    Live use only: feeds and the web are not point-in-time."""
    as_of = as_of or dt.date.today()
    if self.config.provider == "anthropic":
        web = [
            {"type": "web_search_20260209", "name": "web_search", "max_uses": self.config.
    ↪ news_max_searches},
            {"type": "web_fetch_20260209", "name": "web_fetch", "max_uses": self.config.
    ↪ news_max_searches},
        ]
        research = ("Call fetch_news_feeds for the latest headlines from financial news and central-
    ↪ bank\n"
                    "   feeds, then use web_search (and web_fetch on promising URLs when a snippet is
    ↪ not\n"
                    "   enough) to fill gaps and confirm the most market-moving stories.")
    else:
        web = []  # a local model has no web search: the RSS feeds are the source
        research = ("Call fetch_news_feeds for the latest headlines from financial news and central-
    ↪ bank\n"
```

```
                  "   feeds, and pick the stories that matter, judging them from their title and
    ↪ description.")
    return await self._run(
        "news_analyzer", as_of, system=self._fill(NEWS_SYSTEM, as_of, research=research),
        prompt=f"Collect and store the market-moving news for {as_of}.",
        servers=[MCPServerSpec(NEWS_SERVER)], server_tools=web, effort=self.config.effort.get("news", "
    ↪ medium"))

async def analyze(self, as_of: dt.date) -> AgentRun:
    """Stats analyzer: statistics -> saved stats report."""
    return await self._run(
        "stats_analyzer", as_of, system=self._fill(STATS_SYSTEM, as_of),
        prompt=f"Analyze the portfolio assets as of {as_of} and save the stats report.",
        servers=[MCPServerSpec(STATS_SERVER)], effort=self.config.effort.get("stats", "high"))

async def decide(self, as_of: dt.date) -> AgentRun:
    """Execution agent: news + stats -> risk budgets -> target weights."""
    note = BACKTEST_NOTE.format(as_of=as_of.isoformat()) if self.run != "live" else ""
    return await self._run(
        "execution", as_of, system=self._fill(EXECUTION_SYSTEM, as_of, mode_note=note),
        prompt=f"Decide the risk budgets for {as_of} and compute the target weights.",
        servers=[MCPServerSpec(NEWS_SERVER, read_only=True), MCPServerSpec(STATS_SERVER, read_only=True
    ↪ ),
                 MCPServerSpec(PORTFOLIO_SERVER)],
        effort=self.config.effort.get("execution", "high"))
```

7 From decisions to positions

7.1 Budget normalization within bounds

The execution agent proposes positive *relative* budgets $\tilde{b}$. The tool `set_risk_budgets` maps them to budgets with $\sum_i b_i = 1$ and $b_{\min} \le b_i \le b_{\max}$:

1. Let F be the set of fixed assets, initially empty. Scale the free assets to fill the rest: $b_i = \tilde{b}_i \,(1 - \sum_{j \in F} b_j) / \sum_{j \notin F} \tilde{b}_j$.
2. Fix every free asset below $b_{\min}$ at $b_{\min}$, and every one above $b_{\max}$ at $b_{\max}$.
3. Repeat until nothing is out of bounds. If every asset ends fixed, renormalize the fixed values.

This is well defined whenever $n b_{\min} \le 1 \le n b_{\max}$, which validation guarantees. Each pass fixes at least one asset, so it ends after at most n passes. The free assets keep their proposed *ratios*, so the rule changes the agent's intent as little as the bounds allow.

The tool returns the budgets and how far the bounds moved each one. `preview_budgets` shows the same result without saving it.

Code 16: Budget normalization
`../python/nabia/analytics.py:107-139`

```
def project_budgets(budgets: dict[str, float], min_budget: float, max_budget: float) -> dict[str, float
    ↪ ]:
    """Normalizes positive budgets to sum to 1 with every value in [min_budget, max_budget].

    Values beyond a bound are fixed at that bound and the rest rescaled to fill
    the remainder, repeating until nothing is out of bounds.
    """
    if not budgets:
        raise ValueError("no budgets")
    if any(not np.isfinite(v) or v <= 0 for v in budgets.values()):
        raise ValueError("budgets must be positive numbers")
    n = len(budgets)
    if not (min_budget * n <= 1.0 + 1e-12 and max_budget * n >= 1.0 - 1e-12):
        raise ValueError("bounds are infeasible for this number of assets")
    fixed: dict[str, float] = {}
    free = dict(budgets)
```

```
    while True:
        remaining = 1.0 - sum(fixed.values())
        total = sum(free.values())
        scaled = {k: v * remaining / total for k, v in free.items()}
        low = {k for k, v in scaled.items() if v < min_budget - 1e-12}
        high = {k for k, v in scaled.items() if v > max_budget + 1e-12}
        if not low and not high:
            result = {**fixed, **scaled}
            return {k: result[k] for k in budgets}
        for k in low:
            fixed[k] = min_budget
            free.pop(k)
        for k in high:
            fixed[k] = max_budget
            free.pop(k)
        if not free:
            result = {k: fixed[k] / sum(fixed.values()) for k in budgets}
            return result
```

Code 17: The decision tool
`nabia_webui/ai/servers/portfolio.py:45-58`

```
@tool(server)
def set_risk_budgets(as_of: str, budgets: dict[str, float], rationale: str) -> dict:
    """Decide the risk budgets for as_of: one positive number per portfolio symbol
    (relative sizes; normalized to sum to 1 and bounded to [min_budget, max_budget]).
    The rationale should cite the evidence (stats, news) behind each tilt."""
    end = parse_as_of(as_of)
    _check_symbols(ctx, budgets)
    if len(rationale.strip()) < 20:
        raise ValueError("give a rationale that explains the budgets")
    projected = analytics.project_budgets(budgets, ctx.config.min_budget, ctx.config.max_budget)
    ctx.store.save_decision(ctx.run, end, projected, rationale.strip(), decided_by=f"agent:{ctx.config.
    ↪ model_label}")
    changed = {s: round(projected[s] - budgets[s] / sum(budgets.values()), 4) for s in projected}
    return clean({"as_of": end.isoformat(), "budgets": projected,
                  "adjusted_by_bounds": {s: d for s, d in changed.items() if abs(d) > 1e-6}}, 4)
```

7.2 Target weights

`compute_target_weights` loads at least $\max(3W + 30, 120)$ calendar days of prices up to `as_of`. It runs `nabia.latest_weights` with the decided budgets, which is the engine of *The walk-forward portfolio engine* (C++ reference) restricted to the last rebalance. It stores the weights and leverage with the decision.

Each decision records who made it, as `trend_rule` or as `agent:<model>` (for example `agent:ollama:qwen3.8:latest`). Runs on different models can therefore be compared.

Code 18: Budgets to target weights through the C++ engine
`nabia_webui/ai/servers/portfolio.py:82-93`

```
def compute_weights(ctx: ServerContext, as_of: dt.date, budgets: dict[str, float]) -> dict:
    """Runs nabia.latest_weights on prices up to as_of and stores the weights with the decision."""
    import nabia  # the C++ extension, imported on use

    days = int(ctx.config.portfolio.get("covariance_window", 60)) * 3 + 30
    prices = ctx.prices(as_of, days=max(days, 120))
    target = nabia.latest_weights(prices, {**ctx.config.portfolio, "budgets": budgets})
    weights = {s: float(w) for s, w in target.weights.items()}
    ctx.store.save_weights(ctx.run, as_of, weights, target.leverage)
    return clean({"as_of": as_of.isoformat(), "priced_on": target.date.date().isoformat(),
                  "weights": weights, "risk_contributions": target.risk_contributions.to_dict(),
                  "leverage": target.leverage, "converged": target.converged}, 4)
```

7.3 The trend rule baseline

A transparent, model-free policy is the benchmark that agent policies must beat:

$$\tilde{b}_i = \max(1 + s \cdot \mathrm{clip}(m_i, -1, 1),\ 10^{-6}), \qquad s = 0.5,$$

followed by the same bounded normalization. A missing momentum counts as 0. A strong uptrend ($m_i \geq 1$) gets 1.5× the equal budget before normalization, and a strong downtrend 0.5×. Over 2019–2026, on the default universe with monthly decisions, it reached a Sharpe ratio of 0.96 against 0.92 for equal risk.

Code 19: The trend rule
`../python/nabia/analytics.py:94-104`

```python
def trend_tilted_budgets(prices: pd.DataFrame, strength: float = 0.5, min_budget: float = 0.02,
                         max_budget: float = 0.40) -> dict[str, float]:
    """A transparent rule-based policy: equal risk budgets tilted towards assets
    with positive volatility-scaled momentum, bounded and normalized.

    budget_i ~ 1 + strength * clip(momentum_12_1_i, -1, 1). Used as a baseline
    for the agents and as a deterministic decision rule in tests.
    """
    momentum = trend_signals(prices)["momentum_12_1"].fillna(0.0).clip(-1.0, 1.0)
    raw = (1.0 + strength * momentum).clip(lower=1e-6)
    return project_budgets(raw.to_dict(), min_budget, max_budget)
```

Code

trend rule as a backtest policy — `nabia_webui/ai/pipeline.py:64-80`

7.4 Backtest mode

Decision dates. The trading dates in [start, end]: every one for `D`, or the last of each week (`W-FRI`), month (`ME`) or quarter (`QE`).

Policies.

- `trend_rule`;
- `ollama` (agents on the local model);
- `claude` (agents on Claude);
- `agents` (agents on the configured provider).

Procedure.

1. At each date d, the policy decides budgets as of d. A date that already has a decision is skipped, so interrupted backtests resume.
2. The decisions form a budget schedule. Budgets decided at the close of d apply from the rebalance at d on.
3. One continuous backtest runs with the schedule, together with an equal-risk baseline over the same period. The run's policy, period and metrics are stored.

The decision frequency is not the trading frequency: trades follow the parameter `rebalance_frequency`, and each rebalance uses the latest decision.

Code 20: Decision dates
`nabia_webui/ai/pipeline.py:53-61`

```
def decision_dates(index: pd.DatetimeIndex, start: dt.date, end: dt.date, frequency: str = "ME") ->
    ↪ list[dt.date]:
    """Decision dates within [start, end]: every trading date for "D" (daily),
    otherwise the last trading date of each period (pandas offset alias: "ME"
    month end, "W-FRI" weekly, "QE" quarter end)."""
    dates = pd.Series(index, index=index)
    dates = dates[(dates >= pd.Timestamp(start)) & (dates <= pd.Timestamp(end))]
    if frequency.upper() in ("D", "B"):
        return [d.date() for d in dates]
    return [d.date() for d in dates.groupby(pd.Grouper(freq=frequency)).max().dropna()]
```

Code 21: Decisions to a backtest, against equal risk
`nabia_webui/ai/pipeline.py:180-196`

```
def evaluate_decisions(config: AIConfig, run: str, start: dt.date, end: dt.date,
                       prices: pd.DataFrame | None = None) -> AgentBacktest:
    """Backtest of the decisions already stored for ‘run‘ between start and end
    (no new decisions), against equal risk budgets."""
    import nabia

    prices = _backtest_prices(config, start, end) if prices is None else prices
    decisions = [d for d in AgentStore(config.db_url).decisions(run)
                 if start.isoformat() <= d["as_of"] <= end.isoformat()]
    if not decisions:
        raise ValueError(f"run {run!r} has no decisions between {start} and {end}")
    schedule = pd.DataFrame([d["budgets"] for d in decisions],
                            index=pd.DatetimeIndex([d["as_of"] for d in decisions], name="date"))[
    ↪ config.symbols]
    params = dict(config.portfolio)
    result = nabia.backtest(prices, params, budget_schedule=schedule)
    baseline = nabia.backtest(prices, params)
    return AgentBacktest(result, baseline, schedule, decisions)
```

Code

the backtest loop — `nabia_webui/ai/pipeline.py:132-171`
policy to decider — `nabia_webui/ai/pipeline.py:101-105`
agent policy (stats, then execution) — `nabia_webui/ai/pipeline.py:108-115`

Look-ahead control. The tools cannot see the future. In backtests, news comes only from the database (no web access), and the prompt asks the model to treat the decision date as today. One leak cannot be closed by tooling: the language model was trained on data up to its knowledge cutoff and may recognize market history before it. Agent backtests inside that window are therefore optimistic. They are fully out of sample only after the cutoff.

8 News: collection, sentiment and relevance

8.1 Collection

The news analyzer runs once a day in live mode.

1. It calls `query_news` for the last two days to see what is already stored.
2. It gathers candidate stories.
    - `fetch_news_feeds` reads ten public feeds: MarketWatch, CNBC (two), Financial Times, Investing.com, CoinDesk, OilPrice, the Federal Reserve, the ECB and the Bank of England. The list is configurable. The tool parses RSS and Atom, keeps items newer than `max_age_days`, removes URLs already stored, and returns the newest first, with HTML stripped from descriptions. A broken feed is reported without hiding the others.

- On Claude, web search and fetch also fill gaps and confirm stories.

3. It keeps 8–20 material stories, classifies them, and stores them with `store_news_articles`, keyed by URL (an upsert).

Code 22: RSS and Atom headlines without stored articles
`nabia_webui/ai/servers/news.py:71-90`

```
@tool(server)
def fetch_news_feeds(max_age_days: int = 2, max_items: int = 60) -> dict:
    """Latest headlines from the configured financial news RSS feeds (markets,
    economy, central banks, energy, crypto), newest first, skipping articles
    already stored. Use it to find candidate stories to classify and store."""
    cutoff = dt.datetime.now(dt.UTC) - dt.timedelta(days=max_age_days)
    items, errors = [], []
    for source, url in ctx.config.news_feeds.items():
        try:
            items += [i for i in _read_feed(source, url) if i["published"] is None or i["published"] >=
    ↪  cutoff]
        except Exception as error:  # one broken feed must not hide the others
            errors.append(f"{source}: {type(error).__name__}")
    known = ctx.store.known_urls([i["url"] for i in items])
    fresh, seen = [], set()
    for i in sorted(items, key=lambda i: i["published"] or cutoff, reverse=True):
        if i["url"] in known or i["url"] in seen:
            continue
        seen.add(i["url"])
        fresh.append({**i, "published": i["published"].date().isoformat() if i["published"] else None})
    return {"items": fresh[:max_items], "already_stored": len(known), "feed_errors": errors}
```

Code

feed parsing — `nabia_webui/ai/servers/news.py:163-180`

8.2 How sentiment and relevance are produced

Sentiment and relevance are judgements made by the language model for each article. They are not computed by a statistical model or a word list. The news analyzer reads the headline and description, and on Claude possibly the article, then fills the `NewsArticle` schema following the rubric in its prompt:

Sentiment $s \in [-1, 1]$ is the *expected price impact on the affected assets*: -1 clearly bearish, $+1$ clearly bullish, 0 neutral or mixed. It is relative to the assets the story moves, not the tone of the text. A hawkish central bank is bearish for bonds even when the article reads upbeat.

Relevance $r \in [0, 1]$ is the importance of the story for a cross-asset portfolio: 1 for portfolio-wide shocks, about 0.5 for a story that moves one asset class, and lower for minor items.

Classification covers the main topic, the affected asset classes, and the portfolio symbols directly affected.

The server validates what the model produces before storing it:

- the schema bounds $s \in [-1, 1]$ and $r \in [0, 1]$;
- topic and asset classes from fixed lists;
- no future dates;
- symbols from the universe.

Invalid articles are rejected with a reason the model can act on.

Code 23: Article schema
nabia_webui/ai/servers/news.py:27-39

```
class NewsArticle(BaseModel):
    """One news item, as extracted by the news analyzer."""

    url: str = Field(description="Canonical URL of the article (the unique key)")
    published_date: dt.date = Field(description="Publication date, YYYY-MM-DD")
    title: str
    source: str = Field(description="Publisher, e.g. Reuters, Bloomberg, FT")
    topic: Topic = Field(description="Main topic")
    asset_classes: list[AssetClass] = Field(description="Asset classes the news affects")
    symbols: list[str] = Field(default_factory=list, description="Portfolio symbols directly affected")
    sentiment: float = Field(ge=-1.0, le=1.0, description="Price impact on the affected assets: -1
    ↪ bearish .. +1 bullish")
    relevance: float = Field(ge=0.0, le=1.0, description="Importance for a cross-asset portfolio: 0 ..
    ↪ 1")
    summary: str = Field(description="Two or three factual sentences: what happened and why it matters"
    ↪ )
```

Code 24: Validation before storage
nabia_webui/ai/servers/news.py:50-68

```
@tool(server)
def store_news_articles(articles: list[NewsArticle]) -> dict:
    """Store news articles (upsert by URL, so storing an article twice updates it).

    Only store articles with a real URL and publication date you have seen.
    Symbols must be portfolio symbols (see the news_summary universe)."""
    today = dt.date.today()
    universe = set(ctx.config.symbols)
    rejected = []
    accepted = []
    for a in articles:
        if a.published_date > today:
            rejected.append({"url": a.url, "reason": "published_date is in the future"})
        elif not set(a.symbols) <= universe:
            rejected.append({"url": a.url, "reason": f"unknown symbols {sorted(set(a.symbols) -
    ↪ universe)}"})
        else:
            accepted.append(a.model_dump())
    stored = ctx.store.store_news(accepted)
    return {"stored": stored, "rejected": rejected}
```

8.3 Aggregation

`news_summary` takes the articles A published in the d days up to and including `as_of` and groups them by topic, asset class and symbol. An article that affects several classes or symbols counts in each of their groups. For each group g it reports:

- the count;
- the maximum relevance;
- the **relevance-weighted sentiment**

$$S_g = \frac{\sum_{i \in g} s_i \, r_i}{\sum_{i \in g} r_i}.$$

When all relevances in a group are zero, the denominator is replaced by 1, so $S_g = 0$. S_g is a convex combination of the sentiments, so $S_g \in [-1, 1]$. Weighting by relevance lets one portfolio-wide story outweigh several minor ones. The summary also lists the ten most relevant headlines.

The web application's *News* page shows, over the selected filter, article counts per day and the simple (unweighted) average sentiment per topic.

Code 25: Relevance-weighted aggregation
`nabia_webui/ai/servers/news.py:101-134`

```
@tool(server)
def news_summary(as_of: str, days: int = 7) -> dict:
    """Aggregate view of the news in the ‘days‘ days up to as_of: article counts and
    relevance-weighted sentiment per topic, asset class and symbol, plus the top headlines."""
    end = parse_as_of(as_of)
    articles = ctx.store.news(end - dt.timedelta(days=days - 1), end, limit=10_000)

    def aggregate(key_of) -> dict:
        groups: dict[str, list[dict]] = {}
        for a in articles:
            for key in key_of(a):
                groups.setdefault(key, []).append(a)
        out = {}
        for key, items in sorted(groups.items()):
            weight = sum(i["relevance"] for i in items) or 1.0
            out[key] = {
                "articles": len(items),
                "weighted_sentiment": sum(i["sentiment"] * i["relevance"] for i in items) / weight,
                "max_relevance": max(i["relevance"] for i in items),
            }
        return out

    top = sorted(articles, key=lambda a: a["relevance"], reverse=True)[:10]
    return clean({
        "as_of": end.isoformat(),
        "days": days,
        "universe": ctx.config.symbols,
        "articles": len(articles),
        "by_topic": aggregate(lambda a: [a["topic"]]),
        "by_asset_class": aggregate(lambda a: a["asset_classes"]),
        "by_symbol": aggregate(lambda a: a["symbols"]),
        "top_headlines": [{k: a[k] for k in ("published_date", "title", "source", "topic", "symbols",
                                             "sentiment", "relevance", "summary")} for a in top],
    }, 3)
```

Code

article queries — `nabia_webui/ai/servers/news.py:92-99`
storage (upsert by URL) — `nabia_webui/ai/db.py:149-162`
web News aggregates — `nabia_webui/web/app.py:102-121`

8.4 Use and limitations

News is a *secondary* input by design. The execution agent tilts mainly on statistics and uses news to confirm, temper or flag risks the statistics do not show yet. Because the scores are model judgements:

- **They are not reproducible across models**, and only approximately across runs. Decisions record the model that made them.
- **They are uncalibrated.** A sentiment of 0.6 has no defined relation to a return. Using them as quantitative signals would require calibration first, for example a regression of next-day returns on S_g.
- **Coverage depends on the sources.** RSS headlines and descriptions are short, and web access gives fuller context.

Possible improvements:

- an ensemble of scoring models;
- a lexicon or finance-tuned classifier as a reproducible baseline;
- a calibration study of S_g against returns.

Part III
Operation

9 Web application and job runner

`nabia_webui.web` is a FastAPI application with a single-page JavaScript front end (Chart.js). Start it with `python -m nabia_webui.web --config ai_config.json`; it binds to 127.0.0.1 by default.

Endpoint	Purpose
`GET /api/overview`	latest live decision, data coverage, recent jobs, model
`GET /api/news`	articles with filters, counts per day, sentiment by topic
`GET /api/runs`, `/api/runs/{run}/decisions`	runs and their decisions
`GET /api/runs/{run}/stats_report`	a run's stats report as of a date
`GET /api/agent_runs`, `/api/agent_runs/{id}`	agent transcripts (tool calls, usage)
`GET /api/backtests/{run}`	equity, drawdown, metrics, schedule, weights vs equal risk
`GET /api/jobs`, `/api/jobs/{id}`	jobs and their status
`GET /api/jobs/{id}/logs?after=`	job log lines after a given id (tailing)
`POST /api/jobs`	start a job: `download`, `news`, `daily`, `backtest`

Jobs. Jobs run one at a time, in order, in the server's event loop. SQLite allows one writer, and agent runs are costly. Their status is stored in the database. Blocking work runs in worker threads so the server stays responsive:

- downloads;
- the trend rule;
- the final C++ backtest.

Live logs. A logging handler stores every record emitted while a job runs, under that job's id. The id lives in a *context variable*, which follows the job into its asyncio tasks and worker threads, so concurrent web requests never leak into a job's log. The log therefore contains:

- the pipeline's steps;
- each agent turn and each tool call, with input and output previews;
- the model's text and thinking as it streams. Long lines arrive in pieces of about 400 characters, and slow streams every few seconds.

Chatty libraries contribute only warnings and errors. While the job runs, the front end asks every second for the lines after the last one it has, and shows the model's streamed text and thinking in their own styles.

Code 26: Per-job log capture
`nabia_webui/web/jobs.py:36-53`

```
class JobLogHandler(logging.Handler):
    """Stores the log records emitted while a job runs, under that job's id."""

    def __init__(self, store: AgentStore) -> None:
        super().__init__(logging.INFO)
        self.store = store
        self.setFormatter(logging.Formatter("%(message)s"))

    def emit(self, record: logging.LogRecord) -> None:
```

```
        job_id = current_job.get()
        if job_id is None:
            return
        if record.name.startswith(_QUIET) and record.levelno < logging.WARNING:
            return
        try:
            self.store.add_job_log(job_id, record.levelname, record.name, self.format(record))
        except Exception:  # logging must never break the job
            self.handleError(record)
```

Code 27: Running a job with its log context
`nabia_webui/web/jobs.py:106-120`

```
async def _run(self, job_id: int, kind: str, params: dict[str, Any]) -> None:
    async with self._lock:
        token = current_job.set(job_id)
        try:
            self.store.update_job(job_id, status="running", message="started")
            log.info("job #%s: %s %s", job_id, kind, params or "")
            message = await getattr(self, f"_{kind}")(job_id, params)
            log.info("job #%s done: %s", job_id, message)
            self.store.update_job(job_id, status="done", message=message)
        except Exception as error:
            log.exception("job #%s failed", job_id)  # with the traceback, in the job log
            self.store.update_job(job_id, status="failed", error="".join(
                traceback.format_exception_only(error)).strip())
        finally:
            current_job.reset(token)
```

Code

application and endpoints — `nabia_webui/web/app.py:54-217`
job runner — `nabia_webui/web/jobs.py:61-153`
backtest job — `nabia_webui/web/jobs.py:143-153`
front-end log viewer — `nabia_webui/web/static/app.js:204`

10 Verification

pytest (24 tests, offline by default):

- the three MCP servers in process (including the no-look-ahead check);
- the agent loop over real stdio MCP servers with a scripted model (parallel calls, tool errors, paused turns, refusals);
- the Ollama translation and streaming against a mock transport, and live log chunking;
- RSS parsing, the pipeline, and the web API with job logs.

An opt-in test (`NABIA_OLLAMA_TESTS=1`) runs the execution agent end to end on a local model. The analytics the servers use are tested in the Python project.

This document Its excerpts, prompts, tool catalogue and configuration table are extracted at build time. A listed function that no longer exists stops the build.

11 Limitations and future work

- **News scores** are uncalibrated model judgements (§8).
- **Agent backtests** before the model's knowledge cutoff are optimistic.
- **Jobs** run sequentially. A queue with a PostgreSQL backend would allow concurrent jobs.